

- recursively, and then combines the solutions to the sub-problems to solve the original problem. Is it always possible to design a 'Divide and Conquer' algorithm for all problems? If not, what characteristics must a problem exhibit to be amenable to a 'Divide and Conquer' approach? Can you give an example of a problem which is not amenable to this particular approach and explain why it is not suitable?
10. Given a problem, if you are asked to design an algorithm to solve the problem, does the algorithm you design always provide an optimal solution to the problem? Can you give an example of a programming problem, where your solution, while being correct, may not be the optimal one? Can you also give an example of a programming problem where any solution that solves the problem should necessarily be the optimal solution due to the nature of the problem definition?
 11. Can you give a couple of examples of programming problems that can be solved using greedy algorithms? A greedy algorithm makes a sequence of choices as it computes the solution, and for each decision point in the algorithm, the locally best choice (the choice that seems best at that moment) is chosen. Can you give an example of a programming problem for which a greedy solution may not be the optimal solution? What properties must a programming problem typically exhibit in order to have a greedy but optimal solution to it?
 12. How is a dynamic programming solution different from a greedy solution? Are all problems amenable to a dynamic programming solution? What properties must a problem exhibit in order for it to have a dynamic programming solution?
 13. We are all familiar with Minimum Spanning Trees (MST). Given a connected undirected graph with non-negative weighted edges, a spanning tree is a sub-graph which is a tree and that spans all the vertices of the graph. A spanning tree whose sum of edge weights is the least among all spanning trees for a given graph is a minimum spanning tree. A graph can have more than one minimum spanning tree. While we are familiar with Prim's and Kruskal's algorithms for determining MST, these are greedy algorithms. Is it possible to design a dynamic programming algorithm for determining MST? If so, can you provide a dynamic programming algorithm?
 14. How are problems classified into various complexity classes like P, NP and NP-Complete? Give an example of a problem for each complexity class?
 15. Given a 32-bit word, can you write a small program to find the number of ones in the given word? What is the best case and worst case complexity of your solution?
 16. Write a code snippet to reverse a string in place recursively? Can you re-write your solution to make it non-recursive?
 17. Given an arbitrary graph G with V vertices and E edges, and two particular vertices 'u' and 'v' of the graph G, write an algorithm to find whether a path exists between 'u' and 'v'.
 18. Give an example of a programming problem that requires a queue data structure. Can you give an example of a programming problem that requires a stack data structure?
 19. A stack typically supports push and pop operations. Can you design a stack data structure that supports push, pop and find minimum operations?
 20. Given that you are asked to design a dictionary data structure for integer data elements, which is expected to support: (a) insert, and (b) delete, and find operations, please compare the following data structures as potential candidates in terms of time complexity:
 - Unsorted array
 - Sorted array
 - Unsorted singly linked list
 - Sorted singly linked list
 - Unsorted doubly linked list
 - Sorted doubly linked list
 - Binary search tree
 - Hash table
 - Binary heap
 If you know the range of elements you are expected to process, how would your choices change?
 21. What are the common data structures used for representing a graph?
 22. Assume that you are given a singly linked list that does not contain a loop and whose last node has its next field set to NULL. How will you find the node which is Kth node away from the last node on the list?

My 'must-read book' for this month

This month's must-read book suggestion comes from one of our readers, Kavitha Sampath, who recommends the book 'Clean Code' by Robert C Martin. The book has three parts, with Part I containing the principles and practices of writing clean code, Part II containing a number of case studies demonstrating clean code and Part III concluding with advice on how to write clean code. Thanks to Kavitha for her suggestion.

If you have a favourite programming book/article that you think is a must-read for every programmer, please do send me a note with the book's name, and a short write-up on why you think it is useful so I can mention it in this column. This would help many readers who want to improve their software skills.

If you have any favourite programming questions/software topics that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at sandyasm_AT_yahoo_DOT_com. Till we meet again next month, happy programming and here's wishing you the very best! 

By: Sandya Mannarswamy

The author is an expert in systems software and is currently working with Hewlett Packard India Ltd. Her interests include compilers, multi-core and storage systems. If you are preparing for systems software interviews, you may find it useful to visit Sandya's LinkedIn group [Computer Science Interview Training India](http://www.linkedin.com/groups?home=HYPERLINK%20http://www.linkedin.com/groups?home=&gid=2339182%20HYPERLINK%20http://www.linkedin.com/groups?home=&gid=2339182%20gid=2339182) at <http://www.linkedin.com/groups?home=HYPERLINK%20http://www.linkedin.com/groups?home=&gid=2339182%20HYPERLINK%20http://www.linkedin.com/groups?home=&gid=2339182%20gid=2339182>